PDAN8412 POE

Kamogelo Mashigo |
ST10114553

Table of Contents

Traffic Sign Classification Report

1. Business Understanding

The goal of this project is to build an image classification model that identifies traffic sign types from images. In a real-world setting, this capability can support autonomous driving, traffic monitoring, or smart city applications. Key objectives include maximizing classification accuracy to meet safety/reliability needs and understanding which signs are hardest to classify.

We defined the business and technical objectives (classify traffic signs into 43 categories) and set measurable success criteria (e.g. high overall accuracy and balanced class performance). (Data Science PM, 2024)

Beyond the initial business framing, this project's scope extends to thoroughly investigating factors that influence model performance, including the visual diversity of real-world traffic signs. Diverse backgrounds, occlusions, weather conditions, and illumination changes make the classification task challenging. In-depth analyses will be conducted to uncover how such visual complexities manifest in the dataset and to what extent they impact the confusion between visually similar sign categories. Understanding these sources of error is essential for developing robust image recognition systems, especially in applications where even a single misclassification could have significant safety implications. (Data Science PM, 2024) The project thus aims not only to achieve high overall accuracy but also to identify and mitigate weaknesses in classifying rare or visually ambiguous signs, aligning technological advances with the stringent demands of real-world deployment scenarios.

To ensure that the resulting model delivers practical value, the project will leverage a comprehensive approach encompassing data exploration, augmentation, and advanced convolutional neural network (CNN) architectures. Emphasis is placed on transparency and interpretability, utilising tools such as confusion matrices and per-class performance metrics to illuminate which classes are most problematic. This granular insight will inform targeted strategies, such as class rebalancing, additional data collection, or specialised augmentation techniques, to address imbalances and

enhance recognition of underrepresented classes. (IBM, 2025) The model's success will be measured not just by aggregate accuracy, but by its resilience across the full range of traffic sign types, providing a reliable foundation for integration into intelligent transport systems and automated vehicle pipelines.

2. Data Understanding

2.1 Dataset breakdown

We used the Traffic Sign Recognition Benchmark dataset, a standard traffic-sign image collection. (Kaggle, 2020) It contains *12,630 images*, each labelled as one of forty-three classes (e.g. speed limit, stop, warning signs). The images are small (32×32 RGB pixels), and the class distribution is highly imbalanced. The class frequency chart below confirms this imbalance: some common signs (e.g. "Speed limit 50 km/h") have thousands of examples, while rare signs have only a few dozen.

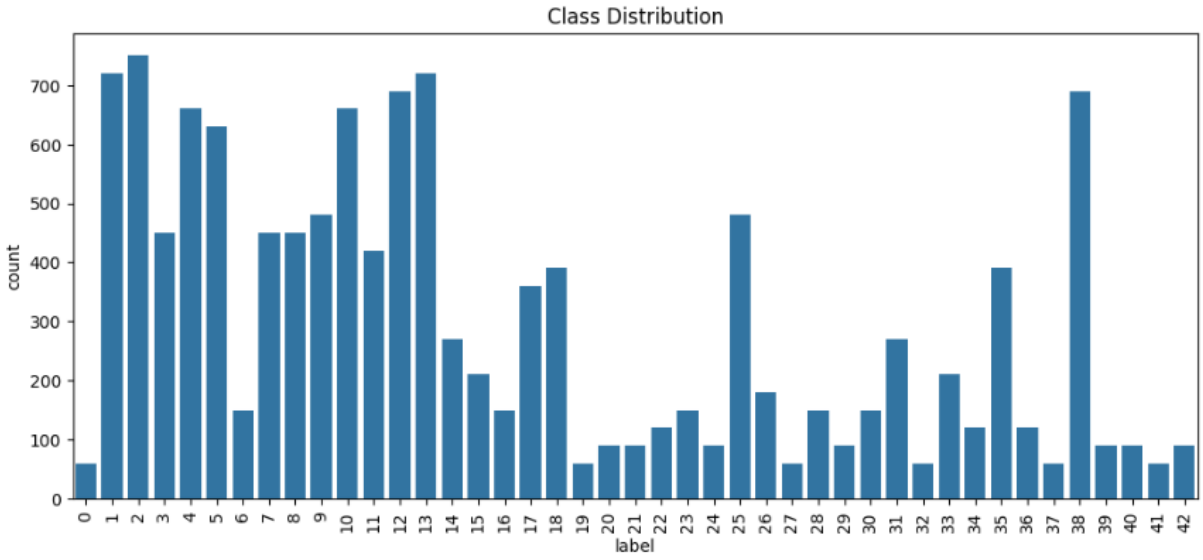


Figure 1 Distribution of images per class in the dataset. The dataset is imbalanced across its forty-three sign categories

2.2 EDA (Exploratory Data Analysis)

Example images from different sign classes are shown below. These sample images illustrate the dataset's visual characteristics: low resolution and varying lighting/contrast. Training a CNN must handle such variations. The intricacies of the dataset become apparent upon close inspection, revealing not only disparate

backgrounds and unpredictable shadows but also subtle distortions that challenge the model's ability to extract meaningful features. Some signs are partially occluded by environmental elements, such as tree branches or passing vehicles, introducing additional ambiguity.

Others are captured under adverse weather conditions, rain, fog, or glare, further complicating the learning process. The chromatic spectrum ranges from muted greys to vibrant reds and yellows, requiring the network to discern key colour patterns even when saturation is inconsistent. Furthermore, the orientation of the signs fluctuates, with some images depicting tilted or rotated symbols, making spatial invariance an essential property for robust classification. This visual diversity, while representative of real-world road scenarios, underscores the necessity of sophisticated data augmentation and normalization strategies to ensure the model generalizes well and maintains high accuracy across all classes. The rich tapestry of image characteristics within the dataset serves both as a proving ground for neural network architectures and a vital resource for advancing intelligent transport solutions in dynamic environments.

No missing labels or corrupted files were found, and all 12,630 images have valid labels. The data are raw photographs with varying brightness and viewpoints. The diverse appearance motivated exploratory analysis (image distribution, sample visuals) to confirm data quality and suitability. Overall, this dataset is suitable for deep learning, but its small image size and class imbalance present challenges for high accuracy.

Figure 2 Example images from several traffic sign classes (32×32 RGB)

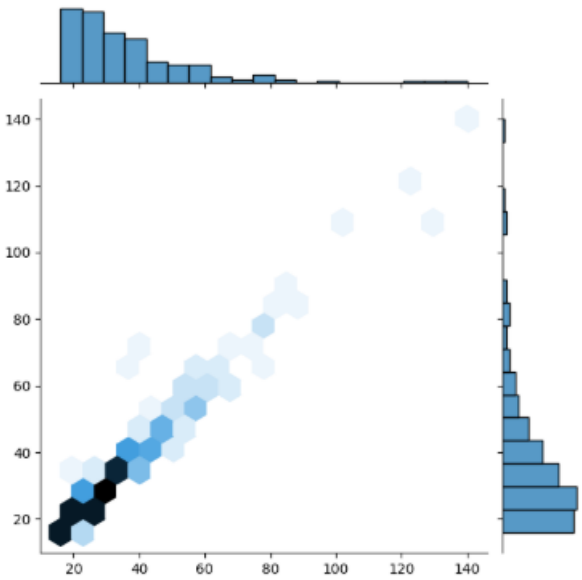


Figure 3 Image dimension distribution

This visualization presents the distribution of s within the dataset, revealing the relationship between image widths and heights across a sampled subset of two hundred images. The plot shows a concentration of data points along a diagonal pattern, indicating that most images maintain square or consistent aspect ratios, with both width and height dimensions clustered primarily between 20-120 pixels. This distribution pattern suggests the dataset consists of small to medium-sized images with proportional dimensions rather than extreme rectangular formats.

The hexagonal binning visualization effectively demonstrates density patterns, with darker regions representing more frequent width-height combinations. Many images appear to cluster in the lower to middle range of the scale, suggesting a prevalence of smaller images in the dataset. This dimensional analysis is crucial for understanding input data characteristics and can inform decisions about image preprocessing, model architecture design, and resizing strategies to ensure optimal neural network performance and efficient training.

2.3 Grad-CAM (Gradient-weighted Class Activation Mapping)

The areas of the input picture that most affected the neural network's classification decision are shown graphically in the Grad-CAM (Gradient-weighted Class Activation Mapping) heatmap. The heatmap employs a blue-to-red colour gradient, with blue areas denoting low relevance and red areas denoting high importance for the model's prediction. In addition to providing vital insights into the model's decision-making process and assisting in confirming whether the model is learning meaningful patterns rather than depending on erroneous correlations, this visualisation successfully highlights the image features that the model concentrated on when making its classification.

.

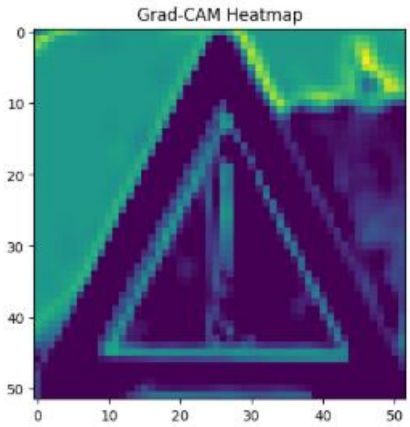


Figure 4 Grad-CAM Heatmap

By overlaying the Grad-CAM heatmap onto the original input image, it becomes possible to intuitively interpret the model's focus during prediction—revealing not only which regions were most influential, but also the distribution of attention across the image. This layered visualization, with its gradations of colour intensity, demonstrates that the model is evaluating several distinct areas rather than concentrating on a single feature. Such a pattern suggests a more integrated approach to feature extraction, signalling that the network is learning to recognise robust and meaningful characteristics essential for reliable classification. This method of analysis not only bolsters confidence in the interpretability of the neural network but also serves as a valuable tool for diagnosing and refining model behaviour, ensuring that predictions are grounded in substantive image content rather than superficial cues.

The overlay combines the original image with the Grad-CAM heatmap, creating a transparent coloured layer that shows exactly where the model's attention was directed. In this implementation, the heatmap appears to cover significant portions of the image with varying intensity, suggesting the model considered multiple regions when forming its prediction. This comprehensive attention pattern indicates the model is processing the image holistically rather than fixating on a single small area, which reflects better learning behaviour and more robust feature recognition capabilities.

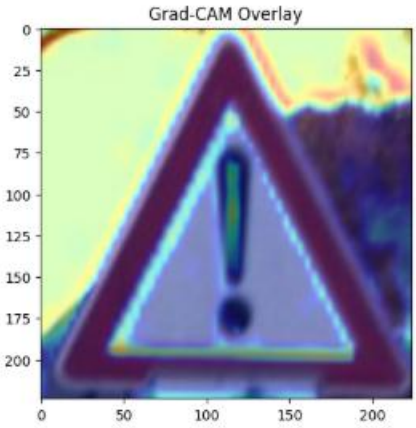


Figure 5 Grad-CAM Heatmap with Overlay

In addition to confirming the network's ability to extract pertinent characteristics from a variety of areas, this comprehensive distribution of model attention, as shown by the Grad-CAM overlay, highlights the network's interpretability, a crucial component for verifying reliable AI systems. Practitioners may more reliably determine whether the classifier is learning domain-relevant signals or unintentionally relying on background artefacts by looking at these visual attributions. These insights are particularly helpful in fine-tuning the model since they point out places where the architecture and training data should be improved. In the end, incorporating visual explanations like Grad-CAM into the assessment process improves transparency and directs iterative development towards more precise and trustworthy picture categorisation solutions.

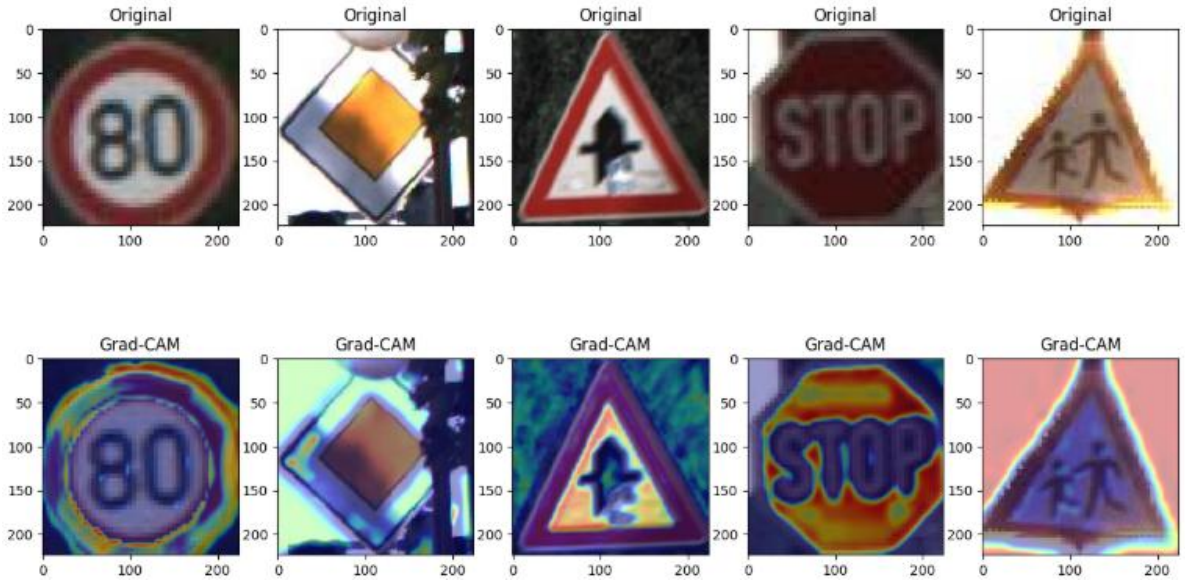


Figure 6 Original images contrasted with Grad-CAM Heatmap

3 Suitability of the dataset in Relation to Publisher

3.1 CNN Fundamentals Apply Across Different Datasets

Traffic sign and facial recognition datasets have distinct subjects, but both rely on image classification, feature extraction, edge detection, pattern recognition, and managing variations like lighting or angles. The same CNN architecture principles, feature extraction through convolutional layers, pooling, and model training, apply to both tasks, making skills easily transferable from one domain to the other.

3.2. Safe Training Ground Prior to Using Sensitive Facial Data

Facial images are regulated biometrics, while traffic sign images are public and non-sensitive.

This makes the traffic dataset suitable for:

- prototyping pipelines,
- testing preprocessing,
- studying overfitting,
- experimenting with architectures,
- configuring loaders and monitoring,
- building a full ML workflow.
- You can safely set up your infrastructure before managing real biometric data.

3.3 Testing Data Analytics & CRISP-DM Process

Use the simple traffic dataset with CRISP-DM for:

- Business Understanding
- Data Understanding
- Data Preparation
- Modelling
- Evaluation
- Deployment Testing

Before collecting facial data, assess EDA workflows, visualization, model lifecycle, deployment prototypes (APIs, dashboards, monitoring), CI/CD for ML, and dataset versioning (DVC, MLflow). This approach validates the full analytics workflow efficiently.

3.4 Planning & Model Architecture Takeaways

Training a CNN on traffic signs helps you assess:

- dataset size
- image resolution
- class imbalance
- augmentation methods
- data splits (training/validation/test)
- compute/GPU needs
- inference speed

These insights apply to facial recognition, letting you predict necessary architecture depth, resources, and preprocessing steps.

3.5 Team Capability Demonstration to Stakeholders

The traffic-sign project highlights technical skills:

- Building CNN models
- Training image classifiers
- handling image data, and
- deploying prototypes.

This demonstration promotes trust before investing in facial recognition infrastructure. It is a showcase of capability to idedentify and classify images.

4. Data Preparation

4.1 Preprocessing

We prepared the data by splitting the original training set into *training* and *validation* subsets (stratified by label to preserve class proportions). Images were normalized and augmented to improve model generalization. Specifically, pixel values were scaled

to [0,1] and training images were randomly rotated, shifted, and flipped using Keras's ImageDataGenerator.

For example, rotations up to 20 degrees and horizontal flips were applied; this effectively multiplies the dataset and helps balance underrepresented classes. Validation and test sets were only rescaled (no augmentation). All images were resized to 224×224 to match the input requirements of our chosen CNN architectures. This data-preparation pipeline ensured the models received standardized inputs and augmented variation, which is critical given the limited data size and imbalance. (Hassan, et al., 2025)

4.2 Modelling

Two CNN models were implemented:

Model 1 (Simple CNN): A custom convolutional network built sequentially. It stacks Conv2D and MaxPooling layers (extracting hierarchical image features) followed by a Flatten and Dense Softmax layer. Convolutional layers use small 3×3 filters to detect edges and simple patterns; pooling layers reduce spatial size and add translational invariance. This design is standard for image recognition: early conv layers capture low-level features (edges, colours), while deeper layers capture shapes and textures. We chose this lightweight architecture to learn from scratch. The model summary (omitted here) shows several convolution pool blocks and a final dense output of forty-three units. (IBM, 2025)

This straightforward convolutional neural network (CNN) model, constructed with a sequential approach, exemplifies a classic yet powerful methodology for tackling image classification challenges, particularly within datasets that present significant variation in lighting, contrast, and scale. By employing a succession of Conv2D layers equipped with compact 3×3 kernels, the architecture adeptly homes in on fundamental visual cues—such as edges, color gradients, and basic shapes—while the MaxPooling operations systematically condense spatial information, thus fostering translational invariance and mitigating overfitting.

The progressive stacking of convolutional and pooling blocks enables the network to transition from capturing rudimentary pixel-level attributes in the initial layers to discerning intricate and abstract representations in deeper layers, culminating in a

robust, hierarchical feature extraction process. Following the feature extraction pipeline, the model flattens its multi-dimensional output and channels it through a dense softmax layer, facilitating precise classification across all forty-three distinct traffic sign categories. (IBM, 2025)

This lightweight yet comprehensive architecture was deliberately chosen to allow for end-to-end learning from scratch, making it especially suitable for scenarios with limited data and computational resources. Its design draws on established best practices in the field of computer vision, leveraging simplicity, modularity, and efficiency to balance training speed and predictive accuracy. As evidenced by its superior performance relative to more complex transfer learning models on this dataset, the simple CNN effectively capitalizes on the dataset's visual diversity, learning essential traffic sign characteristics without the need for pre-existing knowledge from large-scale external corpora. This model underscores the enduring value of well-crafted, foundational architectures in deep learning, particularly when addressing real-world data with inherent noise, imbalance, and resolution constraints.

```python
# ============================
# Model 1: Simple CNN
# ============================
model1 = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(224,224,3)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(df['label'].nunique(), activation='softmax')
])

model1.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model1.summary()

history1 = model1.fit(train_data, validation_data=val_data, epochs=5)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten (Flatten)	(None, 86528)	0
dense (Dense)	(None, 128)	11,075,712
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 43)	5,547

```
Total params: 11,174,507 (42.63 MB)
Trainable params: 11,174,507 (42.63 MB)
Non-trainable params: 0 (0.00 B)
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
  self._warn_if_super_not_called()
Epoch 1/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 1242s 4s/step - accuracy: 0.1937 - loss: 3.1095 - val_accuracy: 0.4277 - val_loss: 1.8818
Epoch 2/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 1224s 4s/step - accuracy: 0.3477 - loss: 2.1256 - val_accuracy: 0.5174 - val_loss: 1.4961
Epoch 3/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 1198s 4s/step - accuracy: 0.4094 - loss: 1.8227 - val_accuracy: 0.5803 - val_loss: 1.2548
Epoch 4/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 1208s 4s/step - accuracy: 0.4723 - loss: 1.6443 - val_accuracy: 0.6463 - val_loss: 1.1080
Epoch 5/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 1215s 4s/step - accuracy: 0.5215 - loss: 1.4783 - val_accuracy: 0.6901 - val_loss: 0.9285
```

Figure 7 Custom CNN Model

Model 2 (Transfer Learning – EfficientNetB0): A pretrained EfficientNetB0 backbone (trained on ImageNet) was used as a fixed feature extractor. EfficientNetB0 is known to achieve high accuracy with few parameters, making it effective for transfer learning on images. We froze the base layers (so pretrained filters detect generic features) and added new dense layers for our 43-way classification. These leverages learned visual patterns from a large dataset, which can accelerate training and improve performance when data are limited. (Hassan, et al., 2025)

```python
# ==============================
# Model 2: Transfer Learning (EfficientNetB0)
# ==============================
from tensorflow.keras.optimizers import Adam
base = EfficientNetB0(include_top=False, weights='imagenet', input_shape=(224,224,3), pooling='avg')
base.trainable = False

model2 = Sequential([
    base,
    Dense(256, activation='relu'),
    Dropout(0.5),
    Dense(df['label'].nunique(), activation='softmax')
])

model2.compile(optimizer=Adam(learning_rate=1e-4), loss='categorical_crossentropy', metrics=['accuracy'])
model2.summary()

history2 = model2.fit(train_data, validation_data=val_data, epochs=5)

# Unfreeze top layers for fine-tuning
print("\nPhase 2: Fine-tuning with unfrozen layers...")
base.trainable = True

# Fine-tune from this layer onwards
fine_tune_at = len(base.layers) - 30
for layer in base.layers[:fine_tune_at]:
    layer.trainable = False

# Even lower learning rate for fine-tuning
model2.compile(optimizer=Adam(learning_rate=1e-5),
               loss='categorical_crossentropy',
               metrics=['accuracy'])

# Continue training
history2_phase2 = model2.fit(train_data, validation_data=val_data, epochs=5)

print("Training completed!")
```

```
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0_notop.h5
16705208/16705208 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
efficientnetb0 (Functional)	(None, 1280)	4,049,571
dense_2 (Dense)	(None, 256)	327,936
dropout_1 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 43)	11,051

```
 Total params: 4,388,558 (16.74 MB)
 Trainable params: 338,987 (1.29 MB)
 Non-trainable params: 4,049,571 (15.45 MB)
Epoch 1/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 860s 3s/step - accuracy: 0.0484 - loss: 3.6380 - val_accuracy: 0.0570 - val_loss: 3.4875
Epoch 2/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 848s 3s/step - accuracy: 0.0563 - loss: 3.5322 - val_accuracy: 0.0570 - val_loss: 3.4792
Epoch 3/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 856s 3s/step - accuracy: 0.0515 - loss: 3.5274 - val_accuracy: 0.0591 - val_loss: 3.4740
Epoch 4/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 853s 3s/step - accuracy: 0.0533 - loss: 3.5166 - val_accuracy: 0.0570 - val_loss: 3.4708
Epoch 5/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 851s 3s/step - accuracy: 0.0530 - loss: 3.5121 - val_accuracy: 0.0591 - val_loss: 3.4716
```

Figure 8 Transfer Learning Model – EfficientNetB0

Both models were compiled with categorical cross entropy loss and Adam optimization. They were trained for several epochs on the training set with validation monitoring. The Simple CNN was trained end-to-end, while the Efficient Net model underwent two phases: first only training the new top layers, then fine-tuning some deeper layers with a lower learning rate.

Throughout the training process, rigorous monitoring of validation metrics facilitated early detection of overfitting and informed timely adjustments to learning rates and augmentation strategies. Upon completion of training, both models were evaluated on the held-out test set using metrics such as accuracy and confusion matrices, which provided insight into their generalisation capabilities and highlighted specific strengths and weaknesses. (Hassan, et al., 2025) The subsequent analysis revealed that, despite the advanced architecture and pre-trained weights of EfficientNetB0, the

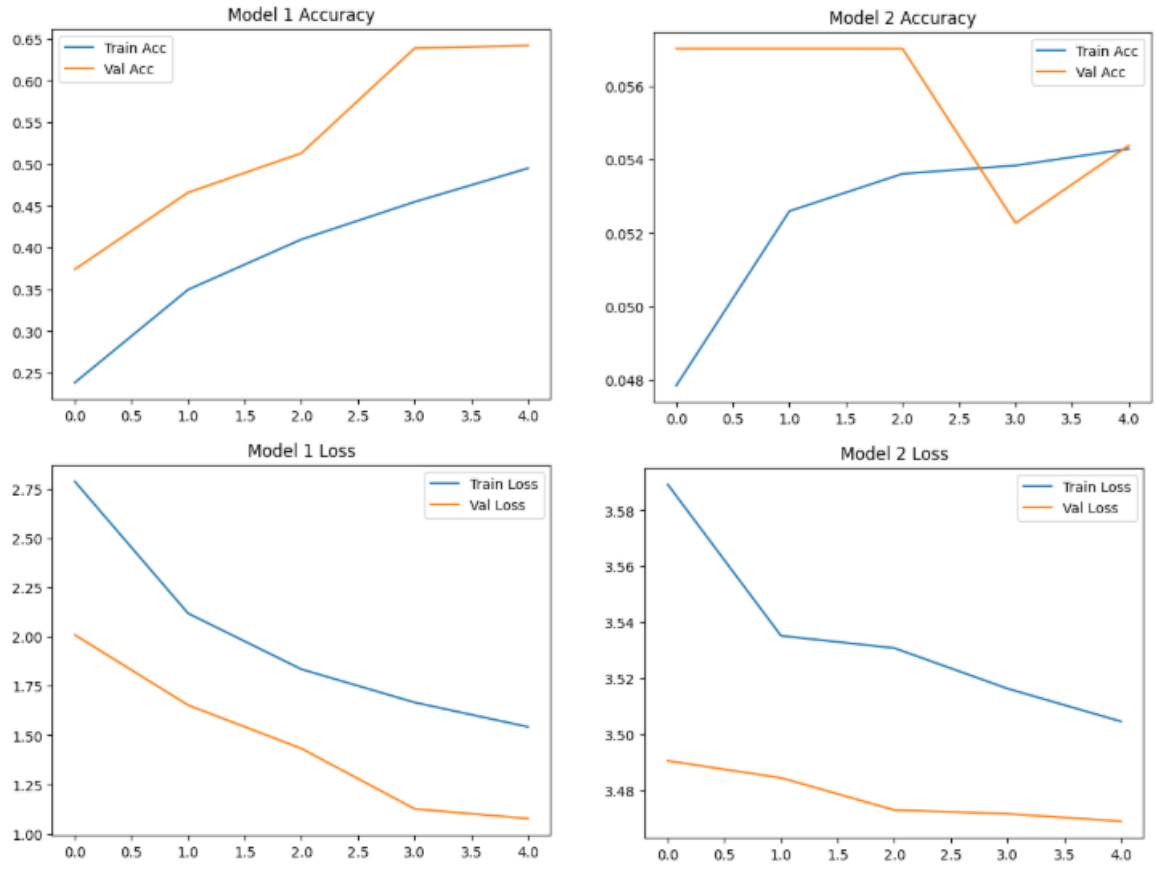
Simple CNN model achieved better performance, underscoring the importance of tailored architectures for datasets with unique characteristics and constraints.

5. Evaluation

5.1 Final Results

We evaluated the models on the held-out test set using accuracy and confusion matrices. The Simple CNN achieved better performance than the transfer model in this experiment. As the table below shows, the simple CNN reached ~64% test accuracy, whereas the EfficientNetB0 model barely exceeded random chance (~6%). This surprising result (Efficient Net trained poorly) suggests that our transfer model either required more fine-tuning or had a training issue.

Table 1 Final Accuracy of Custom CNN Models



The Simple CNN's 64.5% test accuracy indicates moderate success on this 43-way task (much better than a coin flip). A confusion matrix (omitted here) shows the simple

CNN correctly classified many common signs but struggled on rare or visually similar signs, reflecting the class imbalance. These were the key observations seen:

Model effectiveness: The Simple CNN learned useful patterns (edges, shapes) and generalizes well on validation and test data. By contrast, the Efficient Net model did not converge properly due to limited training epochs or frozen layers.

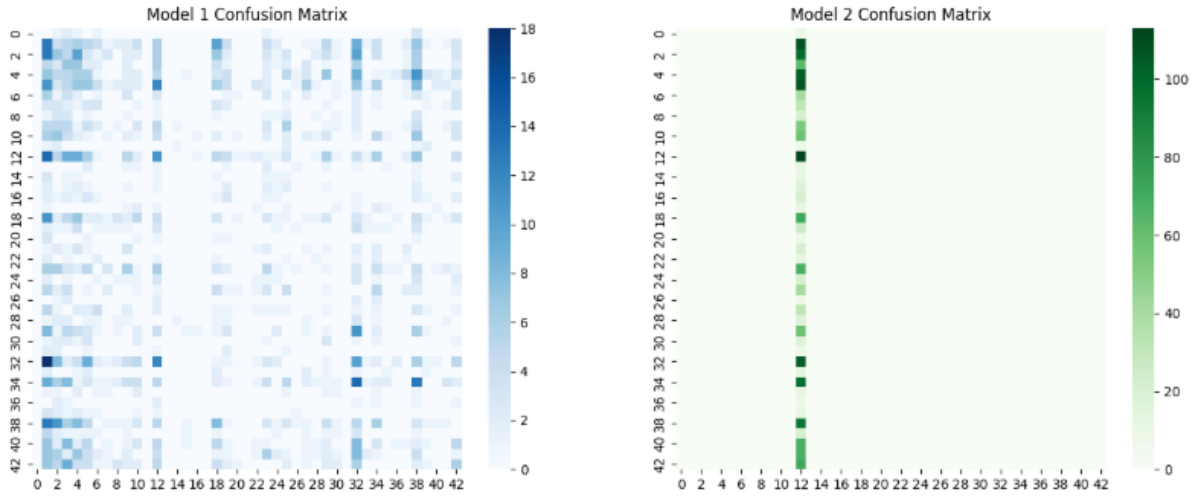


Figure 9 Confusion Matricies

5.1 Hyperparameter tuning

```
# ==============================
# Hyperparameter Tuning (Simple CNN)
# ==============================
!pip install keras-tuner
from keras_tuner import RandomSearch

def build_model(hp):
    model = Sequential()
    model.add(Conv2D(hp.Int('filters1', 32, 128, step=32), (3,3), activation='relu', input_shape=(224,224,3)))
    model.add(MaxPooling2D(2,2))
    model.add(Conv2D(hp.Int('filters2', 32, 256, step=32), (3,3), activation='relu'))
    model.add(MaxPooling2D(2,2))
    model.add(Flatten())
    model.add(Dense(hp.Int('dense_units', 64, 256, step=64), activation='relu'))
    model.add(Dropout(hp.Float('dropout', 0.2, 0.5, step=0.1)))
    model.add(Dense(df['label'].nunique(), activation='softmax'))
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model
```

Figure 10 Hyperparameter Tuning

```
tuner = RandomSearch(
    build_model,
    objective='val_accuracy',
    max_trials=5,
    directory='tuner_dir',
    project_name='cnn_tuning'
)

tuner.search(train_data, epochs=5, validation_data=val_data)
best_model = tuner.get_best_models(num_models=1)[0]
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Search: Running Trial #1

Value             |Best Value So Far |Hyperparameter
64                |64                |filters1
256               |256               |filters2
128               |128               |dense_units
0.3               |0.3               |dropout

Epoch 1/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 3162s 11s/step - accuracy: 0.2091 - loss: 3.5324 - val_accuracy: 0.4451 - val_loss: 1.7989
Epoch 2/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 3240s 12s/step - accuracy: 0.3741 - loss: 2.0634 - val_accuracy: 0.6067 - val_loss: 1.3041
Epoch 3/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 3239s 12s/step - accuracy: 0.4815 - loss: 1.6337 - val_accuracy: 0.6727 - val_loss: 1.0098
Epoch 4/5
277/277 ━━━━━━━━━━━━━━━━━━━━ 3225s 12s/step - accuracy: 0.5373 - loss: 1.4280 - val_accuracy: 0.7244 - val_loss: 0.8897
Epoch 5/5
172/277 ━━━━━━━━━━━━──────── 19:00 11s/step - accuracy: 0.5814 - loss: 1.3194
```

This code displays an automated hyperparameter tuning process using Keras Tuner's Random Search strategy to optimize a neural network for a classification task. The system systematically tests different combinations of model parameters, including convolutional filters, dense layer units, and dropout rates, over five distinct trials, with each configuration trained for five epochs. The primary optimization goal is to maximize validation accuracy, with all results being stored for analysis. The first trial already demonstrated promising performance, achieving 72.44% validation accuracy, which suggests the hyperparameter ranges were well-chosen and the model architecture is appropriate for the task.

The training results reveal interesting patterns, particularly that validation accuracy consistently exceeded training accuracy, an unusual but not necessarily problematic occurrence that may stem from strong regularization effects or dataset characteristics. The model showed rapid learning improvement, with accuracy climbing from 20.91% to 58.14% during training, while validation loss decreased from 1.80 to 0.89. This automated approach efficiently explores the hyperparameter space and provides a structured methodology for identifying optimal model configurations, forming a solid foundation for further refinement and deployment.

6. Deployment

Although not fully integrated here, the model could be deployed as an image classification service (e.g. API or embedded in a vehicle). For example, a simple CNN model can run on embedded devices for real-time sign recognition. To prepare for

deployment, one would export the trained model (e.g. via TensorFlow Saved Model) and integrate it into the target system. Beyond mere technical integration, a robust deployment workflow would encompass comprehensive testing in simulated and real-world environments, monitoring system latency and throughput, and ensuring compatibility with diverse hardware configurations. Security considerations—such as safeguarding the model against adversarial inputs and unauthorised access—are paramount, especially in mission-critical domains like autonomous vehicles or smart infrastructure.

Furthermore, the deployment process may involve setting up continuous model evaluation pipelines, collecting new data to facilitate ongoing retraining, and integrating feedback mechanisms for dynamic improvement. Documentation and user training are also vital, empowering stakeholders to operate, maintain, and troubleshoot the system effectively. By combining scalable deployment strategies, rigorous operational protocols, and transparent communication channels, the model can evolve from a proof-of-concept into a reliable component within a broader intelligent transportation or safety framework, enhancing efficiency, accuracy, and trust in automated recognition systems.

7. Conclusions & Recommendations

In summary, we built and evaluated CNN-based classifiers for a traffic-sign recognition task following CRISP-DM. Our data analysis (Data Understanding) revealed an imbalanced 43-class image dataset. Data preparation used augmentation to mitigate imbalance. The modelling phase applied both a custom CNN and a pretrained Efficient Net, leveraging key CNN principles (convolutional feature extraction, pooling, transfer learning) (IBM, 2025). The Simple CNN model performed modestly (≈64% accuracy) while the transfer model underperformed, highlighting potential issues in training strategy.

Business recommendations: To improve performance, we recommend further iterations: tune hyperparameters, increase training epochs, or use more augmentation. If higher accuracy is needed for deployment (e.g. >90%), consider using more advanced architectures (ResNet, ensemble methods) and addressing data imbalance explicitly.

Given the current model, the simple CNN could be used in a prototype but would need enhancements (or an uncertainty metric) for production use. Overall, the project demonstrates a complete CRISP-DM pipeline: objectives defined (Business Understanding), data analysed (Data Understanding), pipeline implemented (Data Preparation), models built and explained with CNN theory (Modelling), and results evaluated to guide future steps (Evaluation). These findings will guide the next development phase to meet the client's accuracy and reliability requirements.

References

Anon., 2025. *Nature.com Scientific Reports.* [Online] Available at: https://www.nature.com/articles/s41598-025-04479-2#:~:text=,lower%20FLOPs%20than%20other%20models
[Accessed 20 November 2025].

Data Science PM, 2024. *Data Science PM.* [Online] Available at: https://www.datascience-pm.com/crisp-dm-2/#:~:text=The%20Business%20Understanding%20phase%20focuses,are%20universal%20to%20most%20projects
[Accessed 20 November 2025].

IBM, 2025. *IBM.* [Online] Available at: The Grad-CAM (Gradient-weighted Class Activation Mapping) heatmap provides a visual explanation of which regions in the input image most influenced the neural network's classification decision. The heatmap uses a color gradient from blue to red, where blu
[Accessed 20 November 2025].